

美团前端面经

美团前端面试一面

题目

1. 实现一个数组去重函数，要求时间复杂度 $O(n)$
2. 解释事件循环机制中宏任务与微任务的区别
3. 手写Promise.all并处理错误情况
4. CSS实现垂直水平居中的五种方式
5. 解释HTTP/2的多路复用原理
6. React Fiber架构解决了什么问题？

美团前端面试二面

题目

1. 实现深拷贝函数，处理循环引用和特殊对象类型
2. 前端性能优化方案（至少说出10种）
3. 手写二分查找算法
4. Vue3响应式原理与Vue2的区别
5. Webpack优化构建速度的配置方案
6. 解释浏览器缓存机制（强缓存与协商缓存）
7. 设计一个前端错误监控系统

美团前端面试三面

题目

1. 手写LRU缓存算法
2. 实现Promise并发控制函数
3. 解释React Hooks的闭包陷阱及解决方案
4. 前端安全防护方案（XSS/CSRF等）
5. 微前端架构的落地实践方案

6. Node.js事件循环与浏览器事件循环的区别
7. 大型前端项目状态管理方案设计
8. WebAssembly在前端的应用场景

参考答案

美团前端面试一面

1. 实现一个数组去重函数，要求时间复杂度 $O(n)$
2. 使用 `Set`，因为 `Set` 是基于哈希表实现的，查找和插入元素的时间复杂度是 $O(1)$ ，所以可以通过遍历数组并将每个元素加入 `Set` 中来去重，最后返回 `Set` 转为数组。

```
1 function unique(arr) {  
2   return [...new Set(arr)];  
3 }
```

1. 解释事件循环机制中宏任务与微任务的区别

- **宏任务**：是一个任务队列中的任务，通常包括：script代码、`setTimeout`、`setInterval`、I/O任务等。
- **微任务**：是指在当前宏任务执行完之后、下一个宏任务执行之前执行的任务，常见的微任务有 `Promise` 的回调函数、`MutationObserver` 等。

2. 执行顺序：

- 执行一个宏任务；
- 执行所有微任务；
- 然后渲染更新页面；
- 最后再执行下一个宏任务。

3. 手写`Promise.all`并处理错误情况

4. `Promise.all` 接收一个 `Promise` 数组，当所有 `Promise` 都成功时返回一个数组，包含每个 `Promise` 的返回值；若其中有一个 `Promise` 失败，则返回失败的 `Promise`。

```
1 function myPromiseAll(promises) {  
2   return new Promise((resolve, reject) => {  
3     let result = [];  
4     let count = 0;  
5     let len = promises.length;  
6     for (let i = 0; i < len; i++) {
```

```
7     Promise.resolve(promises[i]).then(  
8         value => {  
9             result[i] = value;  
10            count++;  
11            if (count === len) resolve(result);  
12        },  
13        err => reject(err)  
14    );  
15    }  
16    });  
17 }
```

1. CSS实现垂直水平居中的五种方式

◦ Flexbox:

```
1  .container {  
2      display: flex;  
3      justify-content: center;  
4      align-items: center;  
5  }
```

◦ Grid:

```
1  .container {  
2      display: grid;  
3      place-items: center;  
4  }
```

◦ 绝对定位:

```
1  .container {  
2      position: relative;  
3  }  
4  .child {  
5      position: absolute;  
6      top: 50%;  
7      left: 50%;  
8      transform: translate(-50%, -50%);  
9  }
```

- **Inline-block + Text-align:**

```
1  .container {  
2    text-align: center;  
3  }  
4  .child {  
5    display: inline-block;  
6    vertical-align: middle;  
7  }
```

- **Table-cell:**

```
1  .container {  
2    display: table;  
3    width: 100%;  
4    height: 100%;  
5  }  
6  .child {  
7    display: table-cell;  
8    vertical-align: middle;  
9    text-align: center;  
10 }
```

2. 解释HTTP/2的多路复用原理

3. HTTP/2通过“流”的概念将多个请求和响应在同一个TCP连接中并行传输，避免了HTTP/1.x的队头阻塞问题。每个流可以分为多个帧，并行传输，不再依赖请求和响应的顺序，减少了连接建立和网络延迟的开销。

4. React Fiber架构解决了什么问题？

5. React Fiber架构主要解决了更新的优先级调度问题。它使得React的渲染过程可以被打断，允许React在渲染时执行任务的中断和恢复。这样就可以进行异步渲染，提升了性能，避免了阻塞渲染，提升了大应用的响应性。

美团前端面试二面

1. 实现深拷贝函数，处理循环引用和特殊对象类型

2. 深拷贝需要递归处理对象及其属性，并且要处理循环引用。可以使用 `WeakMap` 来存储已拷贝的对象，从而避免循环引用的问题。

```

1  function deepClone(obj, map = new WeakMap()) {
2      if (obj === null || typeof obj !== 'object') return obj;
3      if (map.has(obj)) return map.get(obj);
4      let clone = Array.isArray(obj) ? [] : {};
5      map.set(obj, clone);
6      for (let key in obj) {
7          if (obj.hasOwnProperty(key)) {
8              clone[key] = deepClone(obj[key], map);
9          }
10     }
11     return clone;
12 }

```

1. 前端性能优化方案（至少说出10种）

- 图片优化：使用合适的格式（如WebP），并进行压缩。
- 懒加载（Lazy Loading）：延迟加载图片、视频等资源。
- CSS和JS压缩：减少文件大小，提高加载速度。
- 代码分割（Code Splitting）：按需加载JavaScript代码。
- 使用HTTP/2：提高资源请求的效率。
- 使用CDN：加速资源加载。
- 减少DOM操作：批量更新DOM，避免频繁操作。
- 使用服务端渲染（SSR）：提高首屏渲染速度。
- 采用缓存机制：利用浏览器缓存和服务端缓存。
- 使用WebAssembly：提高计算密集型任务的性能。

2. 手写二分查找算法

```

1  function binarySearch(arr, target) {
2      let left = 0;
3      let right = arr.length - 1;
4      while (left <= right) {
5          let mid = Math.floor((left + right) / 2);
6          if (arr[mid] === target) {
7              return mid;
8          } else if (arr[mid] < target) {
9              left = mid + 1;
10         } else {
11             right = mid - 1;
12         }
13     }

```

```
14     return -1;
15 }
```

1. Vue3响应式原理与Vue2的区别

- **Vue2**: 使用 `Object.defineProperty` 来实现数据的响应式，限制了对数组和对象的某些操作（如新增属性）。
- **Vue3**: 使用 `Proxy` 来实现响应式，能够更好地处理数组、对象的变化，并且性能更高。

2. Webpack优化构建速度的配置方案

- 开启 `parallel` 配置，使用多核构建。
- 使用 `cache` 配置，启用缓存。
- 使用 `DllPlugin`，提前编译第三方库。
- 使用 `hard-source-webpack-plugin`，缓存模块。
- 减少loader和plugin的使用，只在必要时加载。

3. 解释浏览器缓存机制（强缓存与协商缓存）

- **强缓存**: 直接使用缓存内容，避免请求服务器，常见的控制头部是 `Cache-Control` 和 `Expires`。
- **协商缓存**: 浏览器向服务器发送请求，询问缓存是否有效，常见的控制头部是 `Last-Modified` 和 `ETag`。

4. 设计一个前端错误监控系统

- 捕获JavaScript错误: 使用 `window.onerror` 和 `try...catch`。
- 捕获Promise错误: 使用 `window.onunhandledrejection`。
- 将错误信息发送到后端: 使用AJAX或WebSocket发送错误日志。
- 错误日志存储: 在后端数据库中存储错误日志，便于查询和分析。

美团前端面试三面

1. 手写LRU缓存算法

2. LRU缓存需要维护一个有顺序的缓存结构，最近使用的元素放在队头，最久未使用的元素放在队尾。可以通过 `Map` 来实现LRU缓存。

```
1 class LRUCache {
2   constructor(capacity) {
3     this.capacity = capacity;
4     this.cache = new Map();
```

```

5     }
6     get(key) {
7         if (!this.cache.has(key)) return -1;
8         const value = this.cache.get(key);
9         this.cache.delete(key);
10        this.cache.set(key, value);
11        return value;
12    }
13    put(key, value) {
14        if (this.cache.has(key)) this.cache.delete(key);
15        else if (this.cache.size >= this.capacity)
16            this.cache.delete(this.cache.keys().next().value);
17        this.cache.set(key, value);
18    }

```

1. 实现Promise并发控制函数

```

1  function promiseAllLimit(promises, limit) {
2      let index = 0;
3      let result = [];
4      let running = 0;
5      return new Promise((resolve, reject) => {
6          function run() {
7              if (index === promises.length) {
8                  if (running === 0) resolve(result);
9                  return;
10             }
11             if (running < limit) {
12                 const currentIndex = index++;
13                 running++;
14                 Promise.resolve(promises[currentIndex]).then(value => {
15                     result[currentIndex] = value;
16                     running--;
17                     run();
18                 }).catch(reject);
19                 run();
20             }
21         }
22         run();
23     });
24 }

```

1. 解释React Hooks的闭包陷阱及解决方案

2. 闭包陷阱发生在使用React的状态和Effect时，内部的闭包引用了老的状态值。解决方法是使用 `useRef` 或传递函数式更新来获取最新的状态。
3. 前端安全防护方案（XSS/CSRF等）
 - **XSS**：使用 `Content-Security-Policy` (CSP)，对输入内容进行转义，避免将不受信任的内容插入到HTML中。
 - **CSRF**：使用 `SameSite` cookie属性，增加请求的认证信息，使用令牌进行验证。
4. 微前端架构的落地实践方案
5. 微前端的核心思想是将前端应用拆分为多个小的子应用，每个子应用可以独立开发、部署、升级。可以通过 `single-spa`、`qiankun` 等框架来实现。
6. Node.js事件循环与浏览器事件循环的区别
 - **Node.js**：基于libuv库，采用单线程事件循环模型，支持I/O操作的异步处理。
 - **浏览器**：使用浏览器的渲染引擎来处理事件循环，除了I/O事件外，还涉及UI渲染、布局等任务。
7. 大型前端项目状态管理方案设计
8. 使用 `Redux`、`MobX`、`Vuex` 等进行集中式状态管理，确保状态的可预测性。使用 `Context API`、`React Query` 等解决状态的共享和同步问题。
9. WebAssembly在前端的应用场景
10. WebAssembly可以加速计算密集型任务（如图像处理、视频处理、加密算法等），提高Web应用的性能，尤其适用于需要高性能的场景。